# 고객을 세그먼테이션하자 [프로젝트] - 전해진

## 11-2. 데이터 불러오기

### 데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
select *
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
limit 10
```

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

| 행 | InvoiceNo | StockCode | Description | Quantity | InvoiceDate | UnitPrice |
|---|---|---|---|---|---|---|
| 1 | 536365 | 85123A | WHITE HANGING HEART T-LIG... | 6 | 2010-12-01 08:26:00 UTC | |
| 2 | 536365 | 71053 | WHITE METAL LANTERN | 6 | 2010-12-01 08:26:00 UTC | |
| 3 | 536365 | 84406B | CREAM CUPID HEARTS COAT H... | 8 | 2010-12-01 08:26:00 UTC | |
| 4 | 536365 | 84029G | KNITTED UNION FLAG HOT WA... | 6 | 2010-12-01 08:26:00 UTC | |
| 5 | 536365 | 84029E | RED WOOLLY HOTTIE WHITE H... | 6 | 2010-12-01 08:26:00 UTC | |
| 6 | 536365 | 22752 | SET 7 BABUSHKA NESTING BO... | 2 | 2010-12-01 08:26:00 UTC | |
| 7 | 536365 | 21730 | GLASS STAR FROSTED T-LIGHT... | 6 | 2010-12-01 08:26:00 UTC | |
| 8 | 536366 | 22633 | HAND WARMER UNION JACK | 6 | 2010-12-01 08:28:00 UTC | |
| 9 | 536366 | 22632 | HAND WARMER RED POLKA DOT | 6 | 2010-12-01 08:28:00 UTC | |
| 10 | 536367 | 84879 | ASSORTED COLOUR BIRD ORN... | 32 | 2010-12-01 08:34:00 UTC | |

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
select count(*)
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```

쿼리 결과

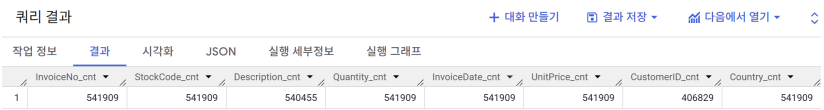
작업 정보 | 결과 | 시각화

| 행 | f0_ |
|---|---|
| 1 | 541909 |

### 데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
select count(InvoiceNo) as InvoiceNo_cnt,
count(StockCode) as StockCode_cnt,
count(Description) as Description_cnt,
count(Quantity) as Quantity_cnt,
count(InvoiceDate) as InvoiceDate_cnt,
count(UnitPrice) as UnitPrice_cnt,
count(CustomerID) as CustomerID_cnt,
count(Country) as Country_cnt
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```




| | InvoiceNo_cnt | StockCode_cnt | Description_cnt | Quantity_cnt | InvoiceDate_cnt | UnitPrice_cnt | CustomerID_cnt | Country_cnt |
|---|---|---|---|---|---|---|---|---|
| 1 | 541909 | 541909 | 540455 | 541909 | 541909 | 541909 | 406829 | 541909 |

# 11-4. 데이터 전처리 방법(1): 결측치 제거

## 컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

```
SELECT
    'InvoiceNo' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
    AS missing_percentage
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
union all
SELECT
    'StockCode' AS column_name,
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
    AS missing_percentage
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
union all
select
    'Description' as column_name,
    ROUND(SUM(CASE WHEN Description is null then 1 else 0 end) / count(*) * 100,2)
    as missing_percentage
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
union all
select
    'Quantity' as column_name,
    ROUND(SUM(CASE WHEN Quantity is null then 1 else 0 end) / count(*) * 100,2)
    as missing_percentage
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
union all
select
    'InvoiceDate' as column_name,
    Round(sum(case when InvoiceDate is null then 1 else 0 end) / count(*) * 100,2)
    as missing_percentage
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
union all
select
    'UnitPrice' as column_name,
    Round(sum(case when UnitPrice is null then 1 else 0 end) / count(*) * 100,2)
```

```
    as missing_percentage
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
union all
select
    'CustomerID' as column_name,
    Round(sum(case when CustomerID is null then 1 else 0 end) / count(*) * 100,2)
    as missing_percentage
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
union all
select
    'Country' as column_name,
    Round(sum(case when Country is null then 1 else 0 end) / count(*) * 100,2)
    as missing_percentage
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부

| 행 | column_name | missing_percenta... |
|---|---|---|
| 1 | InvoiceNo | 0.0 |
| 2 | StockCode | 0.0 |
| 3 | Description | 0.27 |
| 4 | Quantity | 0.0 |
| 5 | InvoiceDate | 0.0 |
| 6 | UnitPrice | 0.0 |
| 7 | CustomerID | 24.93 |
| 8 | Country | 0.0 |

## 결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```
SELECT distinct Description
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
where StockCode = '85123A'
```

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실

| 행 | Description |
|---|---|
| 1 | wrongly marked carton 22804 |
| 2 | WHITE HANGING HEART T-LIG... |
| 3 | CREAM HANGING HEART T-LIG... |
| 4 | ? |

### 결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
DELETE FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
WHERE Description is null or CustomerID is null
```

쿼리 결과

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

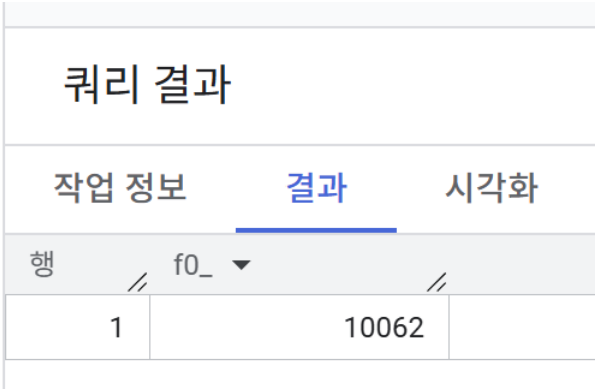
이 문으로 user_data의 행 135,080개가 삭제되었습니다.

## 11-5. 데이터 전처리(2): 중복값 처리

### 중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

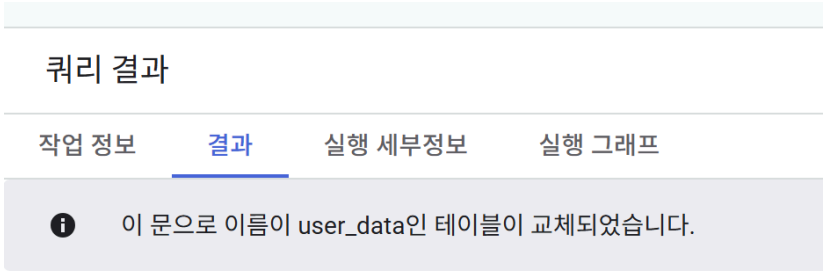
```
select sum(cnt)
from (
          SELECT count(*) as cnt
            from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
        group by InvoiceNo, StockCode, Description, Quantity,
                 InvoiceDate,UnitPrice, CustomerID,Country
          having count(*) > 1
      )
```




### 중복값 처리

- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
  AS
      SELECT *
        from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
    group by InvoiceNo, StockCode, Description,
             Quantity, InvoiceDate,UnitPrice, CustomerID,Country
```

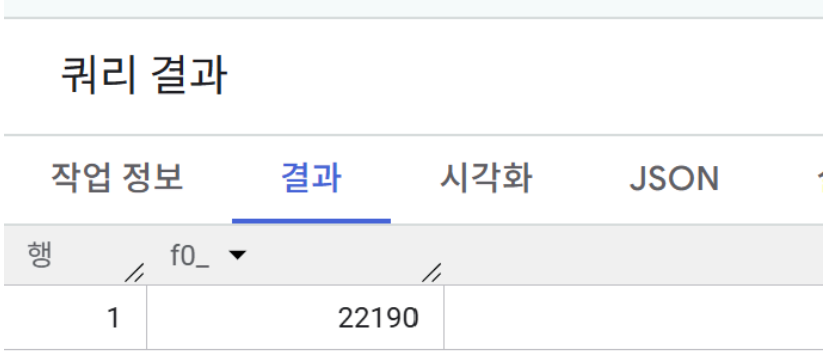



## 11-6. 데이터 전처리(3): 오류값 처리

### `InvoiceNo` 살펴보기

- **고유(unique)한 `InvoiceNo`의 개수를 출력하기**

```
select count(distinct InvoiceNo)
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```

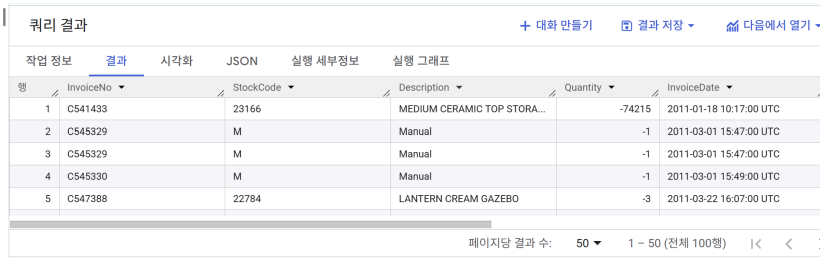



- **고유한 `InvoiceNo` 를 앞에서부터 100개를 출력하기**

```
select distinct InvoiceNo
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
limit 100
```

- **`InvoiceNo` 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)**

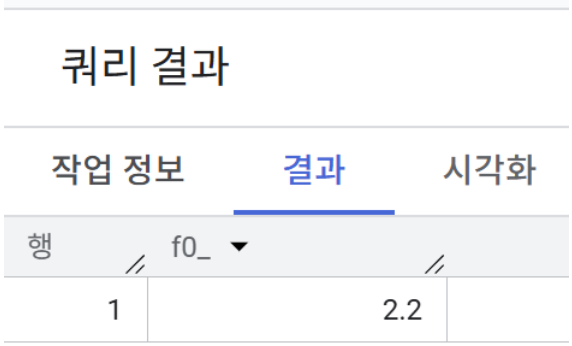
```
select *
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
where InvoiceNo like 'C%'
limit 100
```




- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
select round(
        sum(
```

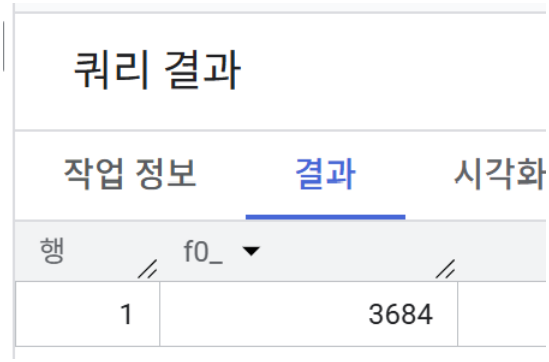
```
        case when InvoiceNo like 'C%' then 1 else 0 end ) / count(*) * 100, 1)
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```




## `StockCode` 살펴보기

- **고유한 `StockCode` 의 개수를 출력하기**

```
select count(distinct StockCode)
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```




- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
select StockCode, COUNT(*) AS sell_cnt
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
group by StockCode
order by sell_cnt desc
limit 10
```

쿼리 결과 + 대화 만들기 결과 저장

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

| 행 | StockCode | sell_cnt |
|---|---|---|
| 1 | 85123A | 2065 |
| 2 | 22423 | 1894 |
| 3 | 85099B | 1659 |
| 4 | 47566 | 1409 |
| 5 | 84879 | 1405 |
| 6 | 20725 | 1346 |
| 7 | 22720 | 1224 |
| 8 | POST | 1196 |
| 9 | 22197 | 1110 |
| 10 | 23203 | 1108 |

페이지당 결과 수: 50 1 – 10 (전체 10행)

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

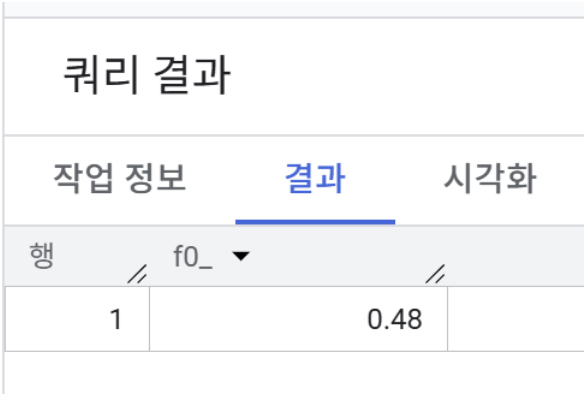
```
SELECT distinct StockCode,
             LENGTH(StockCode) - number_count AS word_cnt
  FROM (
              SELECT StockCode,
                          LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', ''))
                          AS number_count
                 from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
                )
WHERE number_count in (0,1)
```

| 행 | StockCode | word_cnt |
|---|---|---|
| 1 | POST | 4 |
| 2 | M | 1 |
| 3 | C2 | 1 |
| 4 | D | 1 |
| 5 | BANK CHARGES | 12 |
| 6 | PADS | 4 |
| 7 | DOT | 3 |
| 8 | CRUK | 4 |

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

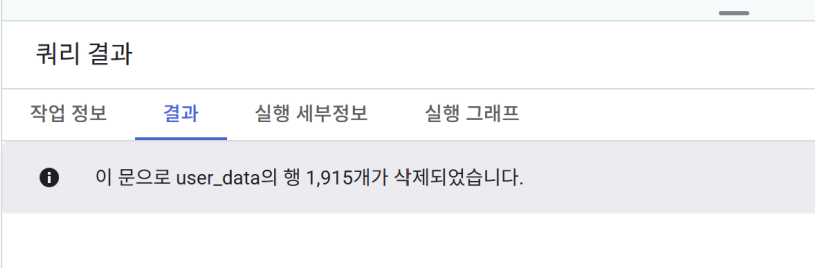
```
SELECT ROUND(
                SUM(
     CASE WHEN LENGTH(StockCode)-LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', ''))
```

```
                    IN (0,1) THEN 1 ELSE 0 END) / COUNT(*) * 100,2)
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```




- **제품과 관련되지 않은 거래 기록을 제거하기**

```
DELETE FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
WHERE StockCode IN (
                                  SELECT distinct StockCode
                                FROM (
                                          SELECT StockCode,
                                                       LENGTH(StockCode)
                                                       - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]',
''))
                                                      AS number_count
                                              from `project-ddf4c901-9232-4ab4-82b.modulabs.user_d
ata`
                                                  )
                                    WHERE number_count in (0,1)
                                       )
```




## `Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
group by Description
```

```
order by description_cnt desc
limit 30;
```

쿼리 결과

+ 대화 만들기 결과 저장

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

| 행 | Description | description_cnt |
|---|---|---|
| 1 | WHITE HANGING HEART T-LIG... | 2058 |
| 2 | REGENCY CAKESTAND 3 TIER | 1894 |
| 3 | JUMBO BAG RED RETROSPOT | 1659 |
| 4 | PARTY BUNTING | 1409 |
| 5 | ASSORTED COLOUR BIRD ORN... | 1405 |

페이지당 결과 수: 50 1 – 30 (전체 30행)

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
DELETE
FROM project_name.modulabs_project.data
WHERE Description in ('Next Day Carriage','High Resolution Image')
```

쿼리 결과

작업 정보 결과 실행 세부정보 실행 그래프

이 문으로 user_data의 행 83개가 삭제되었습니다.

- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
CREATE OR REPLACE TABLE `project-ddf4c901-9232-4ab4-82b.modulabs.user_data` AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```

쿼리 결과

작업 정보 결과 실행 세부정보 실행 그래프

이 문으로 이름이 user_data인 테이블이 교체되었습니다.

## `UnitPrice` 살펴보기

- `UnitPrice` 의 최솟값, 최댓값, 평균을 구하기

```
SELECT MIN(UnitPrice) AS min_price,
             MAX(UnitPrice) AS max_price,
             AVG(UnitPrice) AS avg_price
  FROM project_name.modulabs_project.data;
```

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보

| 행 | min_price | max_price | avg_price |
|---|---|---|---|
| 1 | 0.0 | 649.5 | 2.904956757406... |

- 단가가 0원인 거래의 개수, 구매 수량( `Quantity` )의 최솟값, 최댓값, 평균 구하기

```
SELECT count(quantity) as cnt_quantity,
             min(Quantity) as min_quantity,
             max(Quantity) as max_quantity,
             avg(Quantity) as avg_quantity
    FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
 WHERE UnitPrice = 0
```

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

| 행 | cnt_quantity | min_quantity | max_quantity | avg_quantity |
|---|---|---|---|---|
| 1 | 33 | 1 | 12540 | 420.5151515151... |

- `UnitPrice = 0` 를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.data AS
SELECT *
FROM project_name.modulabs_project.data
WHERE UnitPrice != 0
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

이 문으로 이름이 user_data인 테이블이 교체되었습니다.

# 11-7. RFM 스코어

## Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

```
SELECT DATE(InvoiceDate) InvoiceDay, *
FROM project_name.modulabs_project.data;
```

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

| 행 | InvoiceDay | InvoiceNo | StockCode | Quantity | InvoiceDate | UnitPrice | CustomerID | Country | Description |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 2011-01-18 | 541431 | 23166 | 74215 | 2011-01-18 10:01:00 UTC | 1.04 | 12346 | United Kingdom | MEDIUM CERAMIC TOP S |
| 2 | 2011-01-18 | C541433 | 23166 | -74215 | 2011-01-18 10:17:00 UTC | 1.04 | 12346 | United Kingdom | MEDIUM CERAMIC TOP S |
| 3 | 2010-12-07 | 537626 | 84969 | 6 | 2010-12-07 14:57:00 UTC | 4.25 | 12347 | Iceland | BOX OF 6 ASSORTED COL |
| 4 | 2010-12-07 | 537626 | 84997B | 6 | 2010-12-07 14:57:00 UTC | 3.75 | 12347 | Iceland | RED 3 PIECE RETROSPOT |
| 5 | 2010-12-07 | 537626 | 22497 | 4 | 2010-12-07 14:57:00 UTC | 4.25 | 12347 | Iceland | SET OF 2 TINS VINTAGE E |
| 6 | 2010-12-07 | 537626 | 22195 | 12 | 2010-12-07 14:57:00 UTC | 1.65 | 12347 | Iceland | LARGE HEART MEASURIN |

페이지당 결과 수: 50 1 – 50 (전체 399573행)

- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

```
SELECT DATE(InvoiceDate) InvoiceDay,
            DATE(MAX(InvoiceDate) OVER ()) most_recent_date,
            *
  FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
```

쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

| 행 | InvoiceDay | most_recent_date | InvoiceNo | StockCode | Quantity | InvoiceDate |
|---|---|---|---|---|---|---|
| 1 | 2011-01-18 | 2011-12-09 | 541431 | 23166 | 74215 | 2011-01-18 10:01:00 UTC |
| 2 | 2011-01-18 | 2011-12-09 | C541433 | 23166 | -74215 | 2011-01-18 10:17:00 UTC |
| 3 | 2010-12-07 | 2011-12-09 | 537626 | 84969 | 6 | 2010-12-07 14:57:00 UTC |
| 4 | 2010-12-07 | 2011-12-09 | 537626 | 84997B | 6 | 2010-12-07 14:57:00 UTC |
| 5 | 2010-12-07 | 2011-12-09 | 537626 | 22497 | 4 | 2010-12-07 14:57:00 UTC |
| 6 | 2010-12-07 | 2011-12-09 | 537626 | 22195 | 12 | 2010-12-07 14:57:00 UTC |
| 7 | 2010-12-07 | 2011-12-09 | 537626 | 22771 | 12 | 2010-12-07 14:57:00 UTC |
| 8 | 2010-12-07 | 2011-12-09 | 537626 | 22775 | 12 | 2010-12-07 14:57:00 UTC |
| 9 | 2010-12-07 | 2011-12-09 | 537626 | 22212 | 6 | 2010-12-07 14:57:00 UTC |
| 10 | 2010-12-07 | 2011-12-09 | 537626 | 22774 | 12 | 2010-12-07 14:57:00 UTC |

페이지당 결과 수: 50 1 – 50 (전체 399573행)

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT CustomerID, MAX(DATE(InvoiceDate)) as InvoiceDay
FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
GROUP BY CustomerID
```

쿼리 결과

+ 대화 만들기 결과 저장

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

| 행 | CustomerID | InvoiceDay |
|---|---|---|
| 1 | 12346 | 2011-01-18 |
| 2 | 12347 | 2011-12-07 |
| 3 | 12348 | 2011-09-25 |
| 4 | 12349 | 2011-11-21 |
| 5 | 12350 | 2011-02-02 |

페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM project_name.modulabs_project.data
  GROUP BY CustomerID
);
```

쿼리 결과

+ 대화 만들기 결과 저장 다

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

| 행 | CustomerID | recency |
|---|---|---|
| 1 | 12350 | 310 |
| 2 | 12361 | 287 |
| 3 | 12725 | 371 |
| 4 | 12809 | 168 |
| 5 | 13040 | 8 |
| 6 | 13068 | 10 |
| 7 | 13098 | 1 |
| 8 | 13170 | 26 |

페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-ddf4c901-9232-4ab4-82b.modulabs.user_r`
AS
        SELECT
            CustomerID,
            EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
            FROM (
                        SELECT
                                CustomerID,
                                MAX(DATE(InvoiceDate)) AS InvoiceDay
                            FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
                            GROUP BY CustomerID
                    )
```

작업 정보 결과 실행 세부정보 실행 그래프

이 문으로 이름이 user_r인 새 테이블이 생성되었습니다.

## Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT CustomerID, count(distinct InvoiceNo)
FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
GROUP BY CustomerID
```

| 행 | CustomerID | f0_ |
|---|---|---|
| 1 | 12346 | 2 |
| 2 | 12347 | 7 |
| 3 | 12348 | 4 |
| 4 | 12349 | 1 |
| 5 | 12350 | 1 |
| 6 | 12352 | 8 |
| 7 | 12353 | 1 |
| 8 | 12354 | 1 |

- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT CustomerID, SUM(Quantity)
FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
GROUP BY CustomerID
```

| 행 | CustomerID | f0_ |
|---|---|---|
| 1 | 12346 | 0 |
| 2 | 12347 | 2458 |
| 3 | 12348 | 2332 |
| 4 | 12349 | 630 |

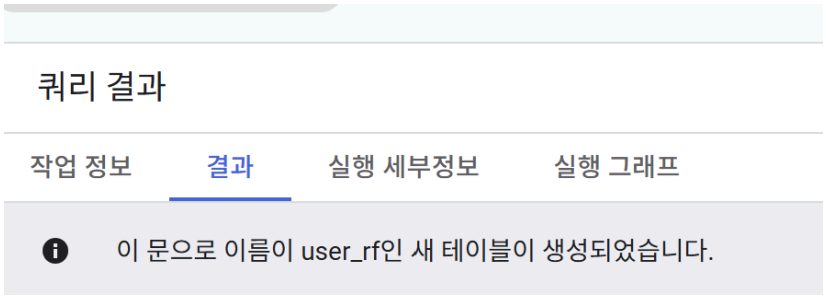
- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE `project-ddf4c901-9232-4ab4-82b.modulabs.user_rf` AS
WITH purchase_cnt AS (
                SELECT CustomerID, count(distinct InvoiceNo) AS purchase_cnt
                    FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
            GROUP BY CustomerID
),

item_cnt AS (
            SELECT CustomerID, SUM(Quantity) item_cnt
                  FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
            GROUP BY CustomerID
)

SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
```

```
JOIN `project-ddf4c901-9232-4ab4-82b.modulabs.user_r` AS ur
  ON pc.CustomerID = ur.CustomerID
```




## Monetary

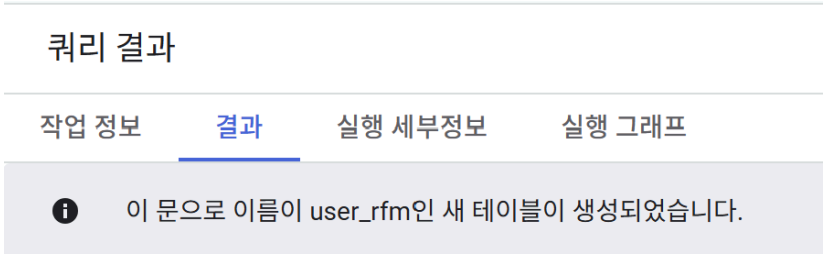
- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
        SELECT CustomerID, ROUND(SUM(UnitPrice*Quantity),1) AS user_total
                  FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
            GROUP BY CustomerID
```

- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt`로 나누어서 3) `user_rfm` 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-ddf4c901-9232-4ab4-82b.modulabs.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ut.user_total / rf.purchase_cnt as user_average
FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_rf` rf
LEFT JOIN (
  -- 고객 별 총 지출액
```

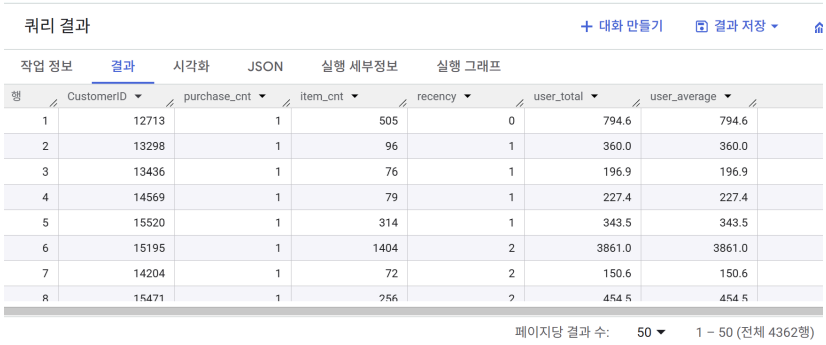
```
  SELECT CustomerID, ROUND(SUM(UnitPrice*Quantity),1) AS user_total
                 FROM `project-ddf4c901-9232-4ab4-82b.modulabs.user_data`
            GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID
```




## RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
select *
from `project-ddf4c901-9232-4ab4-82b.modulabs.user_rfm`
```




| 행 | CustomerID | purchase_cnt | item_cnt | recency | user_total | user_average |
|---|---|---|---|---|---|---|
| 1 | 12713 | 1 | 505 | 0 | 794.6 | 794.6 |
| 2 | 13298 | 1 | 96 | 1 | 360.0 | 360.0 |
| 3 | 13436 | 1 | 76 | 1 | 196.9 | 196.9 |
| 4 | 14569 | 1 | 79 | 1 | 227.4 | 227.4 |
| 5 | 15520 | 1 | 314 | 1 | 343.5 | 343.5 |
| 6 | 15195 | 1 | 1404 | 2 | 3861.0 | 3861.0 |
| 7 | 14204 | 1 | 72 | 2 | 150.6 | 150.6 |
| 8 | 15471 | 1 | 256 | 2 | 454.5 | 454.5 |

# 11-8. 추가 Feature 추출

## 1. 구매하는 제품의 다양성

- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  **2) `user_rfm` 테이블과 결과를 합치기**
  **3) `user_data` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM project_name.modulabs_project.data
```

```
  GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_rfm AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

[결과 이미지를 넣어주세요]

## 2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE ROUND(AVG(interval_), 2) END AS average_inter
val
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate), DAY)
AS interval_
    FROM
      project_name.modulabs_project.data
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_data AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```

[결과 이미지를 넣어주세요]

## 3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  **1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수**
  **2) 취소 비율(cancel_rate) :  각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율**
  - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기**
    **(취소 비율은 소수점 두번째 자리)**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS

WITH TransactionInfo AS (
  SELECT
```

```
    CustomerID,
    # [[YOUR QUERY]] AS total_transactions,
    # [[YOUR QUERY]] AS cancel_frequency
  FROM project_name.modulabs_project.data
  # [[YOUR QUERY]]
)

SELECT u.*, t.* EXCEPT(CustomerID), # [[YOUR QUERY]] AS cancel_rate
FROM `project_name.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

# 회고

[회고 내용을 작성해주세요]

Keep :

Problem :

Try :